Machine Learning Methods

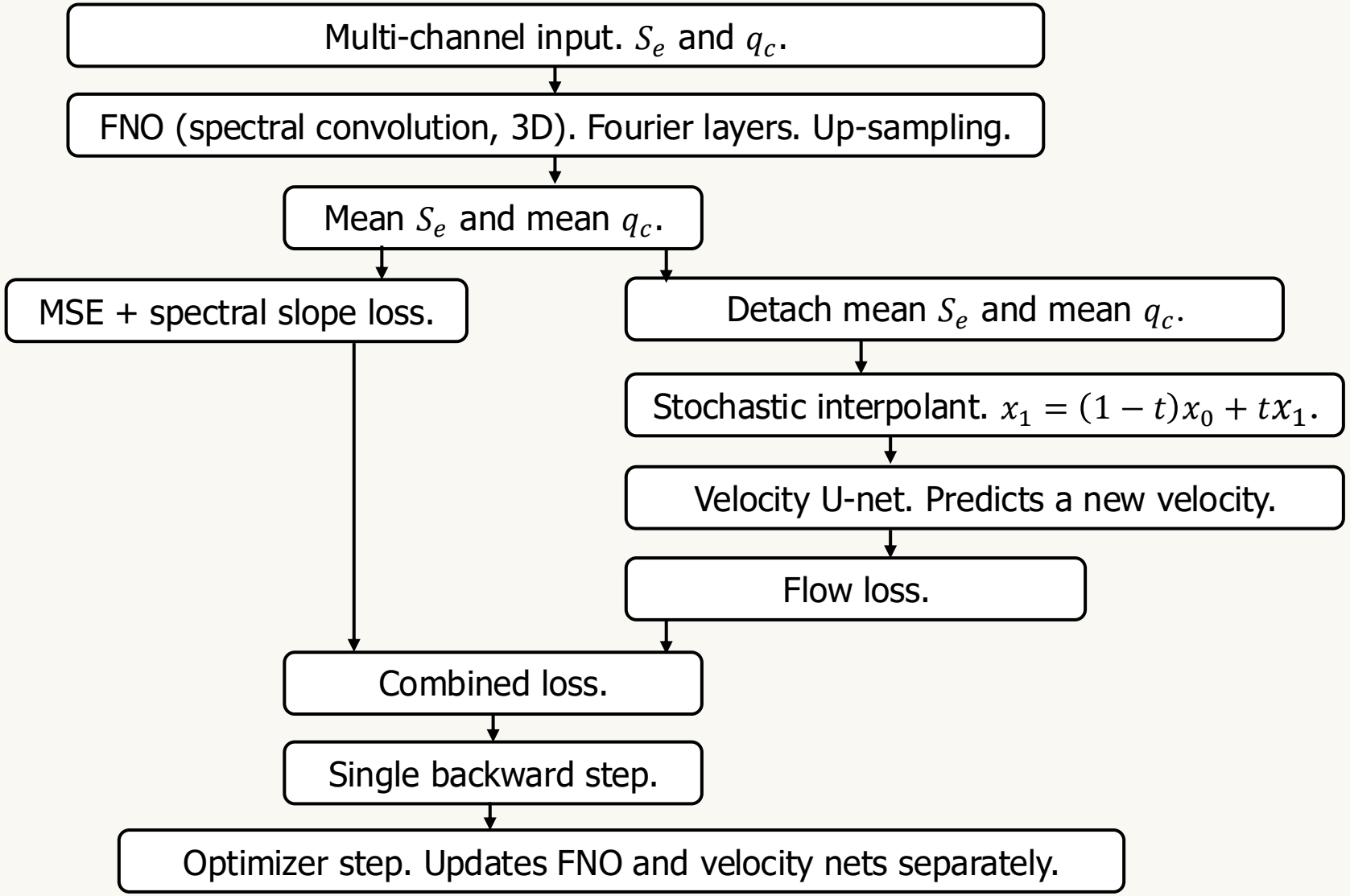
Emulation of turbulence → **FNO**

Fourier neural operators (FNO) are resolution-invariant neural surrogates that learn solution operators for partial differential equations (PDE) directly in the Fourier space. They are well-suited to emulate the large-scale, deterministic structure of turbulent flow.

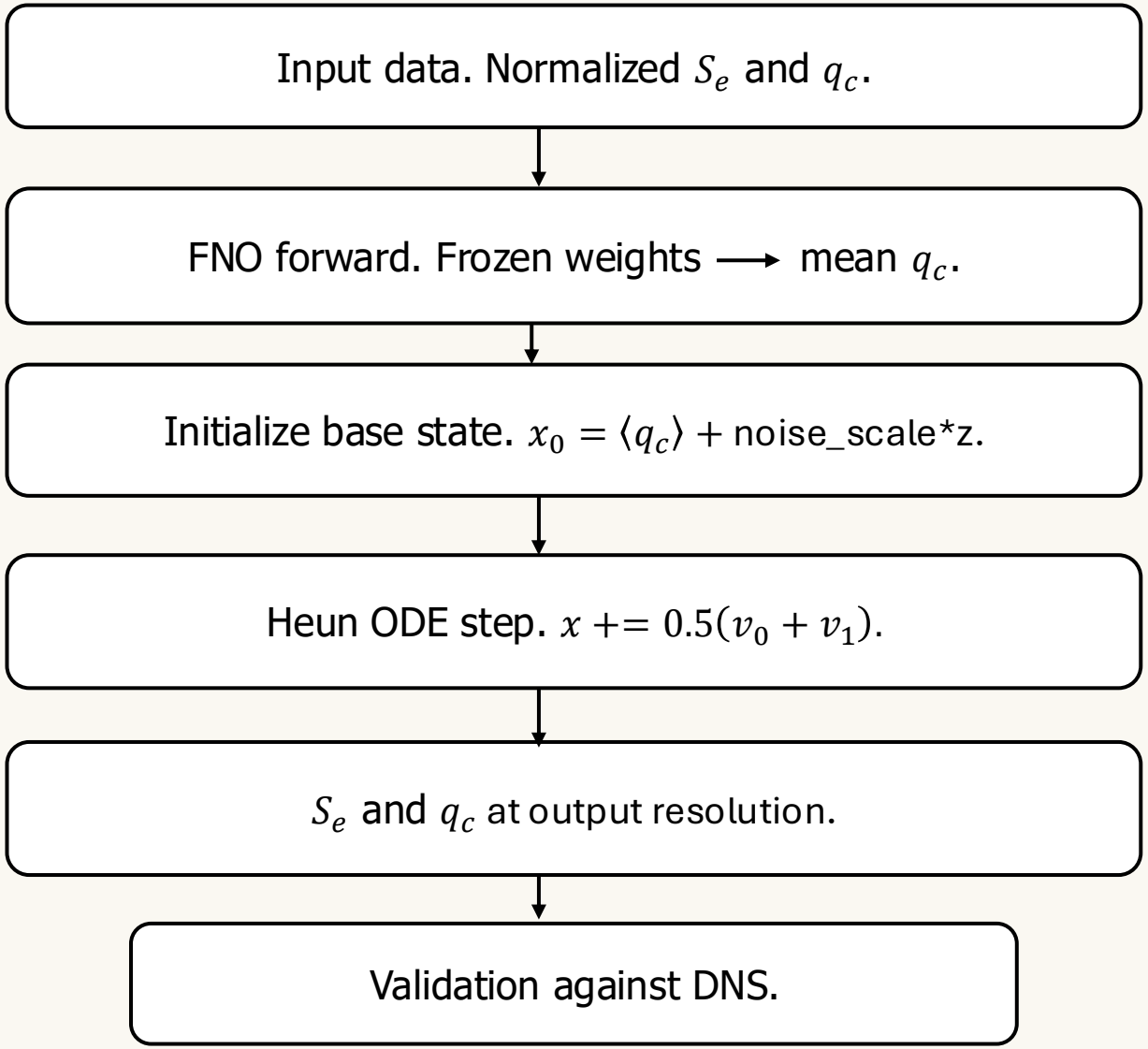
Emulation of microphysics → **SI**

Stochastic interpolants (SI) are generative models that transport samples along a learned velocity field to bridge the gap between FNO-predicted mean field and true instantaneous field. They are well-suited to emulate the small-scale, stochastic fluctuations of cloud microphysics.

FNO + SI Model: Training



FNO + SI Model: Inference



FNO Architecture

- **Spectral convolution layers:** Truncate the lowest Fourier modes. Apply learned complex weights. Transform back the core FNO operator.
- **Spectral zero-padding:** Pads the field in Fourier space from 128^3 to 192^3 by filling the high-wavenumbers with zeros. It gives resolution-independent output size.
- **Lifting layer:** Lifts $[Se, q_c]$ to a higher dimensional latent space through a $1 \times 1 \times 1$ convolution.
- **Fourier layers:** Applies the global spectral convolution alongside the local $1 \times 1 \times 1$ convolution, adds them, and passes through Gaussian error linear unit (GELU).
- **Projection:** The local linear transformations project the high-dimensional latent features to the desired physical outputs.

Forward Pass

Input (2, 128^3). S_e and q_c.

↓

Convolution of S_e and q_c.

↓

Spectral zero-padding. 128^3 → 192^3

↓

Lifting layer. Fourier layers.

↓

Projection.

↓

Output (2, 192^3). $\langle S_e \rangle$ and $\langle q_c \rangle$.

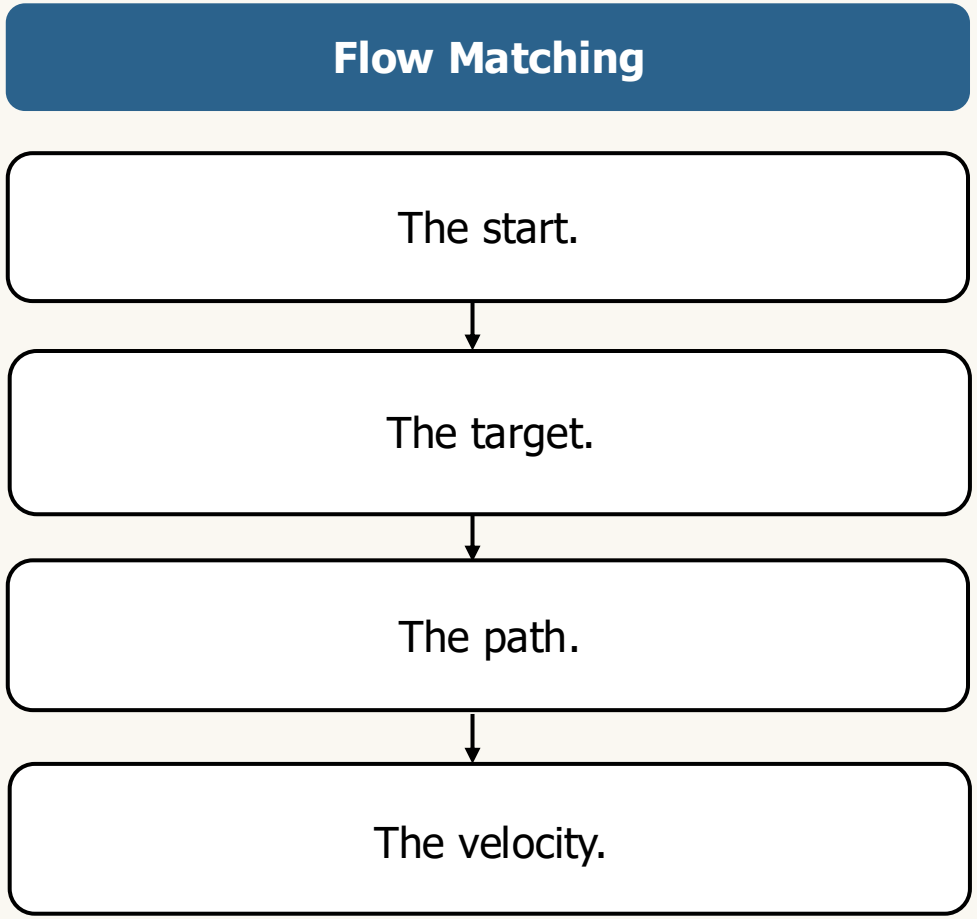
FNO Architecture : Spectral Convolution

1) Fast Fourier transform (FFT) of input data over 3 dimensions. The data is transformed from physical space to frequency space.
2) Keep only the 6^3 lowest frequency modes and truncate the high frequency modes. The frequency domain has a conjugate symmetry. Because half of the data is mathematically redundant, only 4 of the 8 3D quadrants (octants) are need to capture the full picture.
3) The network applies a linear transformation (a weight matrix) directly to these low frequency modes.
4) Once the frequency data is updated by the learned weights, an inverse FFT is applied to get data in 3D physical space.

Weights per octant: 4 octants $\times$ 2 layers $\times$ $(4 \times 4 \times 6 \times 6 \times 6 \times 2)$ = 55296 spectral weights. Inside the parenthesis, 4 → input channels, 4 → output channels, 6^3 → modes, 2 → complex number.

SI Architecture

- **The start:** A small amount of noise (noise_scale*z) is added to the FNO-predicted mean to get the starting point x_0.
- **The target:** The actual high-resolution ground truth data is the target x_1.
- **The path:** The framework defines a straight line between the start and the target: $x_t = (1-t)x_0 + tx_1$.
- **The velocity:** Because the path is a straight line, the 'velocity' required to move from start to target is constant: $v = x_1 - x_0$.



SI Architecture: Velocity Network

1) Encoder: This is the left side of the U-shape and is the contracting path. The input spatial size shrinks but the number of channels (features) increases.
2) Bottleneck: This is the very bottom of the U-shape and connects encoder with decoder. It operates at the lowest spatial resolution.
3) Decoder: This is the right side of the U-shape and is the expanding path. It up-samples the abstract features, decreases the number of channels and returns the original resolution.
4) Skip connections: These are used to concatenate the up-sampled feature map (currently in the decoder) and the high-resolution feature map (saved from the encoder). The fused output is processed by the next convolution.

Time conditioning: In flow matching, the network (also called U-net) is learning a time-dependent velocity field. Hence, a time-dependent bias is added directly to the activations of encoder.

Joint Training and Backward Pass

- The input passes through the FNO. PyTorch records the mathematical operations into a computational graph called **Graph A**. The FNO's specific loss is calculated.
- The **.detach()** command takes the numerical output of the FNO but severs its connection to Graph A. It creates a new tensor that has the exact same data but tells Python's autograd engine (that does backpropagation) not to track gradients past this point.
- During flow matching, PyTorch builds a new computational graph called **Graph B** for the U-net. The U-net's specific loss is calculated.
- The two separate losses are added and the **total.backward()** function traces the gradients from U-net's loss perfectly backward through Graph B to update the U-net, without reaching the FNO. Similarly, the gradients from FNO's loss are traced independently back through Graph A.
- PyTorch handles the independent graphs in parallel.

[Backpropagation calculates the gradient (the direction and magnitude) of the prediction error with respect to every weight in the network, and uses these gradients to adjust the weights, minimizing future errors.]

Important Parameters

FNO modes = **6**

FNO width = **4**

FNO layers = **2**

U-net channels = **2**

Time dimension = **32**

FNO Parameter Count = **55449**

U-net Parameter Count = **3931**

Input resolution = $\mathbf{128^3}$

Output resolution = $\mathbf{192^3}$

Noise scale = **0.1**

Learning rate = **1e-3**

Flow steps = **20**

Epochs = **300**

Slope weight = **0.3**

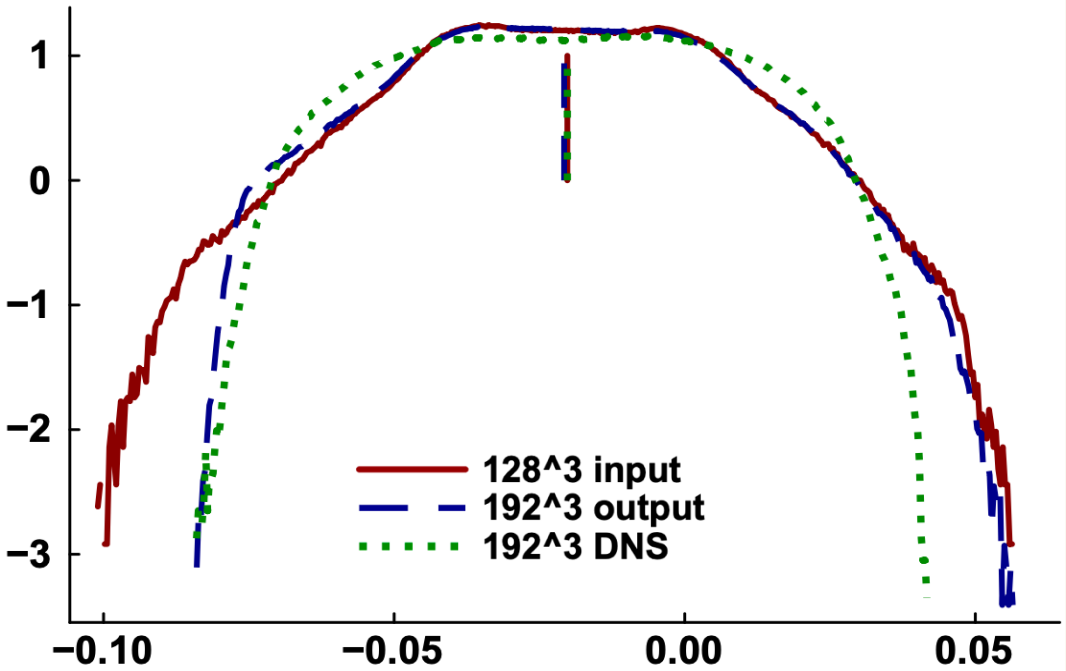
Memory Requirements

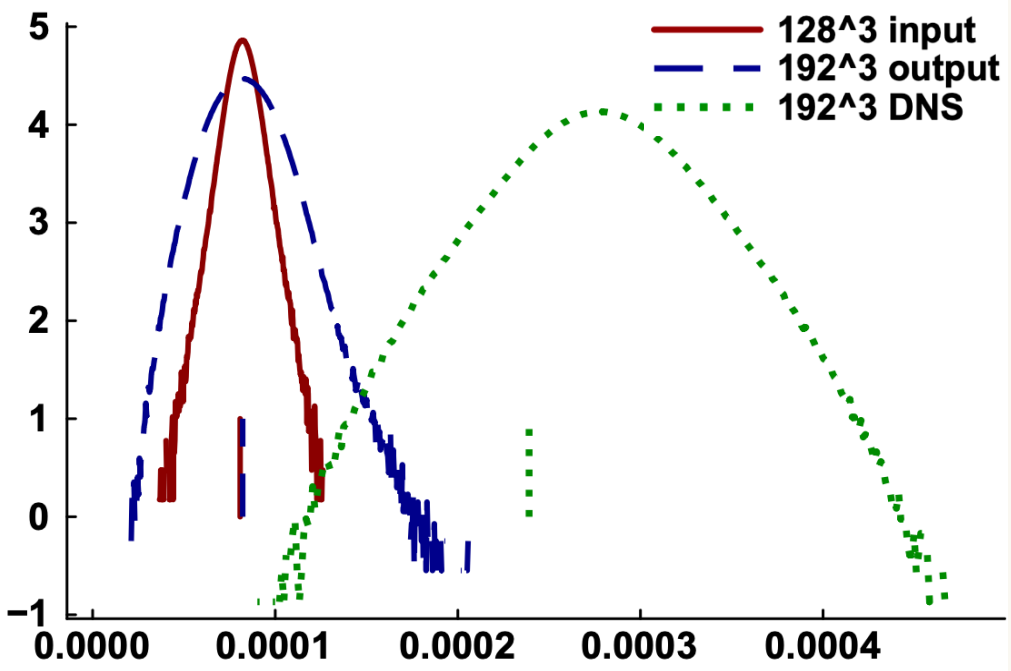
Breakdown of the memory budget	
FNO activations (width=4, layer=2, points=192^3)	~ 3.6 GB
U-net activations (base channels=2, points=192^3)	~ 1.1 GB
Training data (pairs=2^3=8, Float64 in Julia)	~ 1.15 GB
Overhead for optimizer + framework	~ 1.5 GB
Total	~ 7.35 GB

[By design, activations are more memory-intensive than learning weights because weights are sized by the channel/mode counts, while activations scale with N^3 and backpropagation needs simultaneous access to intermediate activation tensors.]

Results from Run #1

The PDFs of 192^3 output are compared against the 128^3 input and 192^3 DNS ground truth. The vertical line segments are the respective mean values.





PDFs and means of the S_e. The blue profile tracks the green profile across the whole range.

PDFs and means of the q_c. The blue profile is nowhere near the green profile.

Results from Run #1

The prediction of cloud water micing ratio (q_c) failed because:

1) **The training target was inaccurate:** The 192^3 target was just a spectral zero-padding of the 128^3 input, or in other words, an unsampled copy with no real sub-grid structures. The true $2.4 \times q_c$ amplification was never in the data.
2) **One loss let S_e starve q_c:** A single pooled MSE over both channels let high-variance S_e dominate the shared gradient. Sparse q_c minimized its loss by predicting its own mean, with no incentive to learn the small-scale fluctuations.

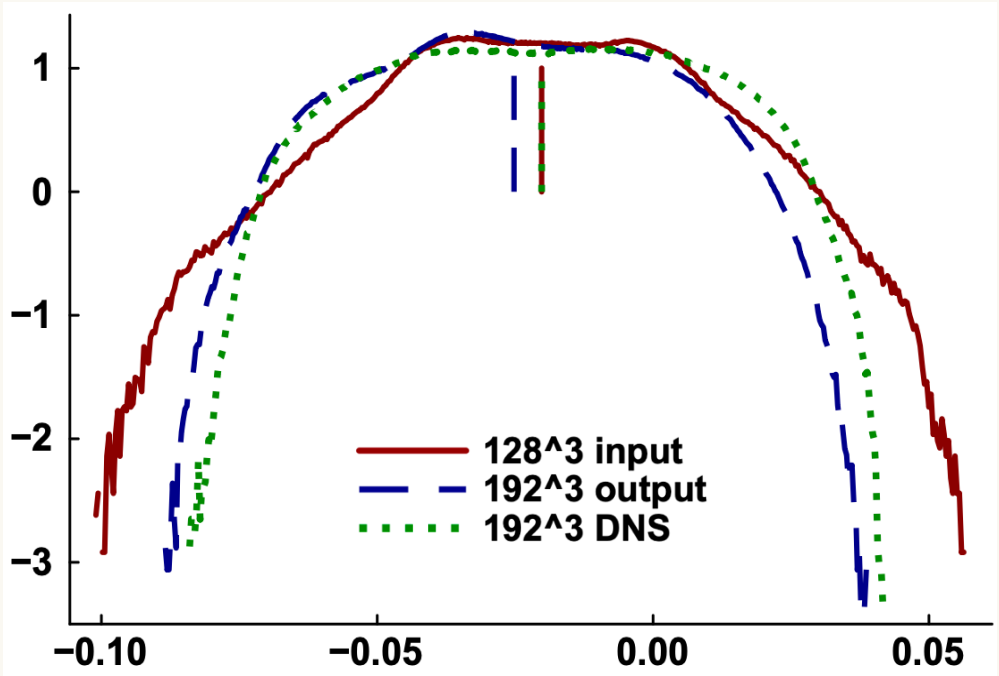
Neither cause is fixable by tuning the network as they are related to data and objective.

Changes Made in Run #2

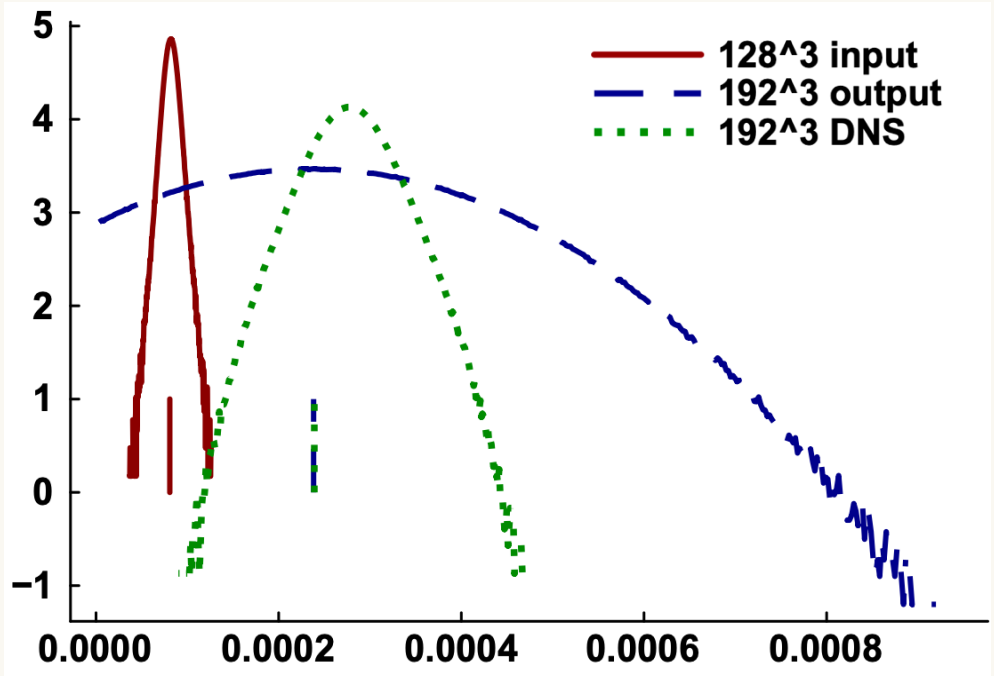
Aspect	Run #1	Run #2
Training target	Spectral zero-padding of 128^3 input (No new information)	True 192^3 DNS fields
Loss	One pooled MSE over both channels	Per-channel weighted MSE (S_e and q_c are separate)
Normalization	Starts from 128^3 only	Pooled across 128^3 + 192^3
Noise scale	0.1 for both channels	0.1 for S_e and 0.3 for q_c
FNO modes/widths/layers	6/4/2	6/4/2

Results from Run #2

The PDFs of 192^3 output are compared against the 128^3 input and 192^3 DNS ground truth. The vertical line segments are the respective mean values.



PDFs and means of the S_e.



PDFs and means of the q_c.

Comparison of Run #1 and Run #2

Metric	Error in Run #1	Error in Run #2	Verdict
Mean S_e vs DNS truth	0.027	0.247	Worse
Fluctuating S_e vs DNS truth	0.093	0.106	Unchanged
Mean q_c vs DNS truth	0.65	0.0033	Significantly Better
Fluctuating q_c vs DNS truth	0.86	0.37	Better

For future runs, the stochastic FNO model's width/modes/layers need to be increased. This will increase the memory requirements and thus the model needs to be deployed on Perlmutter.